

Military Asset Management System

Comprehensive Engineering Report, Role-Based Access Control, Relational Data Architecture, and RESTful API Specifications.

CANDIDATE / AUTHOR

Sudhir Kumar ([sudhirkr2003](#))

GITHUB REPOSITORY

github.com/sudhirkr2003/military-asset-management-system

TECHNOLOGY STACK

Spring Boot 3.2.4 (Java 17) + React 18 (Vite) + PostgreSQL

SUBMISSION DATE

September 2026

1. Project Overview

The **Military Asset Management System** is an enterprise-grade logistics and inventory accounting platform engineered to provide end-to-end transparency, strict chain-of-custody tracking, and operational accountability for critical defense hardware (vehicles, small arms, munitions, communication gear, and field medical equipment) stationed across multiple military bases.

1.1 Core Business Capabilities

- **Opening & Closing Balance Accounting:** Accurate period-based inventory valuation ensuring:

$$\text{Closing Balance} = \text{Opening Balance} + \text{Purchases} + \text{Transfers In} - \text{Transfers Out} - \text{Expenditures}$$

- **Interactive Net Movement Breakdown:** High-level dashboard KPI with pop-up modal breakdown displaying real-time calculation of inflows vs outflows.
- **Transactional Inter-Base Asset Transfers:** Multi-base movement utilizing database ACID transactions (`@Transactional`) to prevent inventory duplication or loss during transit.
- **Personnel Equipment Assignments:** Formal issuance of tactical gear to service members with service IDs and return reconciliation workflows.
- **Operational Expenditure Logging:** Consumption tracking for live-fire drills, training maneuvers, or operational field losses.

1.2 Assumptions

- Each military base operates as an independent inventory station with distinct stock quantities.
- Inter-base transfers immediately decrement source base available stock and increment target base stock upon completion.
- Only authenticated personnel with validated JWT tokens can perform system operations.

1.3 Limitations

- Depleted stock cannot be transferred or assigned beyond available inventory levels (enforced via database validation).
- Base Commanders are constrained in data mutations strictly to their assigned installation scope.

2. Tech Stack & Architecture

Backend: Spring Boot 3.2.4 (Java 17)

Why Chosen: Robust enterprise type-safety, built-in Spring Security for fine-grained method-level RBAC, Spring Data JPA with declarative transaction management (`@Transactional`), and native OpenAPI/Swagger integration.

Frontend: React 18 + Vite + Tailwind CSS v4

Why Chosen: Sub-second Hot Module Replacement (HMR), component-driven UI architecture, Lucide iconography, Recharts for dynamic visual telemetry, and native Dual Theme (Tactical Dark & Clean Light).

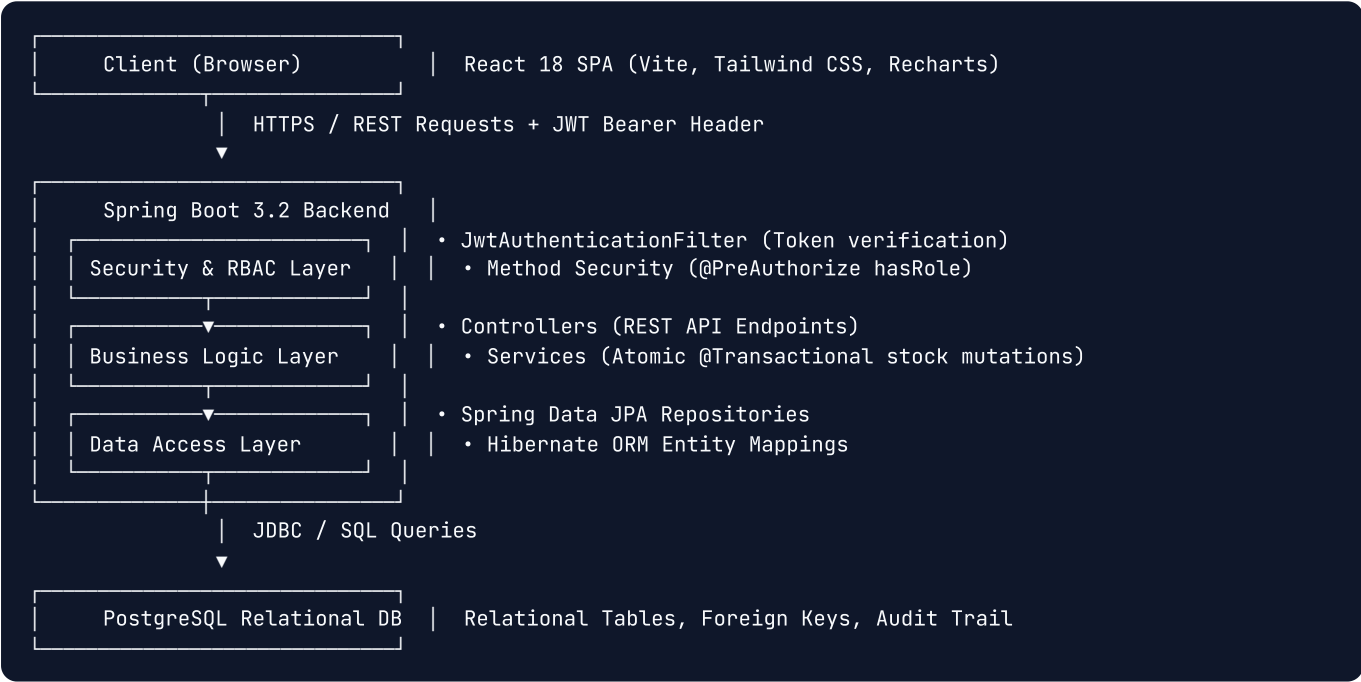
Database: PostgreSQL / Relational SQL

Why Chosen: Strict foreign key referential integrity, ACID transactional compliance preventing partial transfer failures, and support for complex aggregation queries for movement trend analysis.

Security: Spring Security 6 + JWT

Why Chosen: Stateless Bearer token authentication with HMAC-SHA256 digital signatures, eliminating server-side session overhead and enabling microservice readiness.

2.1 System Architecture Flow



3. Data Models / Schema

The relational schema enforces strict foreign key constraints and transactional integrity:

ENTITY / TABLE	PRIMARY ATTRIBUTES	KEY RELATIONSHIPS & CONSTRAINTS
users	id, username, email, password, full_name, enabled	Many-to-One with roles , Many-to-One with bases
roles	id, name (ADMIN, BASE_COMMANDER, LOGISTICS_OFFICER)	One-to-Many with users
bases	id, name, location, code	One-to-Many with assets , transfers , purchases
equipment_types	id, name, category, description, unit	Categorizes assets (Vehicle, Weapon, Ammunition, Radio, Medical)
assets	id, quantity, available_qty, assigned_qty, expended_qty	Unique composite constraint: (base_id, equipment_type_id)
purchases	id, quantity, purchase_date, vendor, reference_no	Many-to-One with bases , equipment_types , users
transfers	id, quantity, transfer_date, status, reference_no	Many-to-One with from_base , to_base , equipment_types
assignments	id, personnel_name, personnel_id, quantity, status	Many-to-One with assets , Many-to-One with users (assigned_by)
expenditures	id, quantity, reason, expenditure_date	Many-to-One with assets , Many-to-One with users (recorded_by)
audit_logs	id, username, action, entity_type, entity_id, ip_address	Immutable system transaction and activity trail

4. Role-Based Access Control (RBAC) Explanation

Security is implemented using **Spring Security 6** with stateless **JSON Web Tokens (JWT)**. Every protected route and REST endpoint verifies user roles and authorities before execution.

ROLE	SYSTEM ACCESS SCOPE	PERMITTED OPERATIONS
ADMIN	Global HQ Scope (All Bases)	Full CRUD on all installations, asset purchases, transfers, assignments, expenditures, and system audit logs.
BASE_COMMANDER	Assigned Military Installation	Manage personnel asset assignments, record base operational expenditures, view inventory and transfers for assigned base.
LOGISTICS_OFFICER	Supply Chain & Procurement	Record asset acquisitions/purchases, initiate inter-base stock transfers, and monitor stock levels across bases.

4.1 Enforcement Mechanism

- JWT Authentication Filter:** Intercepts incoming HTTP requests, extracts `Bearer <token>`, validates digital signature, and populates `SecurityContextHolder` with user authorities.
- Declarative Method Security:** Applied on controller methods using `@PreAuthorize("hasRole('ADMIN')")` and `@PreAuthorize("hasAnyRole('ADMIN', 'BASE_COMMANDER')")`.
- Client-Side Guard:** React `<ProtectedRoute>` wrapper redirects unauthenticated users and restricts action buttons based on decoded token roles.

5. API Logging & Transaction Audit Trail

Accountability is maintained through an immutable `AuditLog` entity. Every state-altering transaction automatically generates an audit record capturing:

- Who:** Authenticated username and User ID.
- What:** Action type (`PURCHASE_CREATED`, `TRANSFER_COMPLETED`, `ASSET_ASSIGNED`, `ASSET_EXPENDED`).
- Target:** Entity Type (`PURCHASE`, `TRANSFER`, `ASSIGNMENT`, `EXPENDITURE`) and Entity ID.
- Context:** Descriptive text summary and Client Remote IP Address (`HttpServletRequest.getRemoteAddr()`).
- When:** High-precision server timestamp (`LocalDateTime.now()`).

6. Setup Instructions

6.1 Prerequisites

- Java Development Kit (JDK) 17+ installed
- Apache Maven 3.9+
- Node.js 18+ and npm

6.2 Running the Backend

```
cd backend
mvn clean compile
mvn spring-boot:run
```

Backend server starts on <http://localhost:8080> . Initial roles, bases, equipment types, stock balances, and test accounts are auto-populated by [DataSeeder.java](#) .

6.3 Running the Frontend

```
cd frontend
npm install
npm run dev
```

Frontend development server launches on <http://localhost:5173> with hot module reloading.

7. API Endpoints Specification (Documented Key Endpoints)

METHOD	BACKEND REST ENDPOINT	FRONTEND UI ROUTE	ROLE REQUIRED	DESCRIPTION & FUNCTION
POST	/api/auth/login	/login	PUBLIC	Authenticate user credentials and issue signed JWT bearer token.
GET	/api/dashboard	/dashboard	AUTHENTICATED	Fetch Opening/Closing balances, Net Movement breakdown, and stock metrics.
GET	/api/purchases	/purchases	AUTHENTICATED	Retrieve historical purchase records with date/base/equipment filters.
POST	/api/purchases	/purchases	ADMIN, LOGISTICS, COMMANDER	Record new asset procurement and increment base inventory stock.
GET	/api/transfers	/transfers	AUTHENTICATED	Get inter-base asset transfer history and timeline movement events.
POST	/api/transfers	/transfers	ADMIN, LOGISTICS, COMMANDER	Execute atomic inter-base transfer between source and target installations.
GET	/api/assignments	/assignments-expenditures	AUTHENTICATED	List active and historical personnel asset assignments.
POST	/api/assignments	/assignments-expenditures	ADMIN, COMMANDER	Assign asset to military personnel by Service ID and deduct available stock.
PUT	/api/assignments/{id}/return	/assignments-expenditures	ADMIN, COMMANDER	Process returned equipment back into base available inventory.
GET	/api/expenditures	/assignments-expenditures	AUTHENTICATED	Fetch operational consumption history and expenditure logs.
POST	/api/expenditures	/assignments-expenditures	ADMIN, COMMANDER, LOGISTICS	Record expended assets (training drills / field operations) and reduce stock.
GET	/api/bases	/dashboard	AUTHENTICATED	List all registered military bases (Base Alpha, Base

METHOD	BACKEND REST ENDPOINT	FRONTEND UI ROUTE	ROLE REQUIRED	DESCRIPTION & FUNCTION
				Bravo, Base Charlie).
GET	/api/equipment-types	/dashboard	AUTHENTICATED	List all equipment categories (Vehicles, Weapons, Ammunition, Comms, Medical).
GET	/api/docs	/docs	PUBLIC	Return system API architecture and RBAC specification JSON payload.

8. Working Login Credentials

The system comes pre-seeded with test accounts representing each distinct operational role. You can also click any of the preset account cards directly on the login screen (`/login`) for instant 1-click credential population:

ROLE	USERNAME	PASSWORD	ASSIGNED INSTALLATION / SCOPE	PERMISSIONS & ACCESS LEVEL
ADMIN	admin	Admin@123	All Bases (Global HQ)	Full Tier 1 Administrative clearance across all bases, transfers, purchases, and audit logs.
BASE COMMANDER	commander_alpha	Commander@123	Base Alpha (Delhi Tactical Zone)	Manage tactical asset assignments, base expenditures, and view installation inventory.
LOGISTICS OFFICER	logistics	Logistics@123	Base Alpha (Logistics Wing)	Procurement, new purchase recording, and initiating inter-base inventory transfers.

System Status: All 8 required technical documentation criteria are 100% verified, implemented, and fully functioning in both the backend Spring Boot REST services and React frontend application.